

MONGODB COMMAND

◇ Database Operations

Show all databases

```
show dbs
```

Make a database the current working database

```
use mydb
```

◇ Collection Operations

Create a new collection

```
db.createCollection("student")
```

Show all collections

```
show collections
```

Remove a collection

```
db.student.drop()
```

◇ Inserting Documents

Insert a single document

```
db.student.insert({name: "Arya", rollno: 1})
```

Insert multiple documents

```
db.student.insertMany([
  {name: "Arya", rollno: 1},
  {name: "saumya", rollno: 2},
  {name: "tanu", rollno: 3}
])
```

◇ Updating Documents

Update a single document

```
db.student.update({rollno: 1}, {$set: {name: "yash"}})
```

Update multiple documents

```
db.student.updateMany({class: "FY"}, {$set: {batch: "2025"}})
```

Upsert (Update if exists, else Insert)

```
db.student.update({_id: 3}, {$set: {name: "Def"}}, {upsert: true})
```

Add a new field

```
db.student.updateMany({}, {$set: {age: 21}})
```

Remove a field

```
db.student.update({name: "Anil"}, {$unset: {sub2: 1}})
```

◇ Querying Documents

Find all documents

```
db.student.find()
```

Formatted output

```
db.student.find().pretty()
```

Find with a condition

```
db.student.find({name: "Arya"})
```

Find with specific fields

```
db.student.find({}, {name: 1, age: 1, _id: 0})
```

Using relational operators

```
db.student.find({rollno: {$gt: 5}})
```

Using logical operators

```
db.student.find({$or: [{age: 42}, {sub1: "English"}]})
```

◆ Deleting Documents

Delete all matching documents

```
db.student.remove({age: 42})
```

Delete only one matching document

```
db.student.deleteOne({age: 42})
```

Delete multiple documents

```
db.student.deleteMany({age: 42})
```

◆ Counting Documents

Count all documents

```
db.student.count()
```

Count with condition

```
db.student.count({age: 35})
```

New version count

```
db.student.countDocuments()
```

◇ **Pagination and Sorting**

Limit results

```
db.student.find().limit(3)
```

Sort documents ascending

```
db.student.find().sort({name: 1})
```

Sort documents descending

```
db.student.find().sort({salary: -1})
```

Skip documents

```
db.student.find().skip(2).limit(3)
```

◇ **String Pattern Matching (Regex)**

Begins with 'M'

```
db.student.find({name: /^M/})
```

Ends with 's'

```
db.student.find({name: /s$/})
```

Contains 'e'

```
db.student.find({name: /e/})
```

◇ Arrays in Documents

Insert document with array

```
db.student.insert({rollno: 1, name: "Arya", subject: ["Math", "Science", "English"]})
```

Find documents with array element

```
db.student.find({subject: {$in: ["Math", "English"]}})
```

Array size

```
db.student.find({subject: {$size: 3}})
```

Update array value at index

```
db.student.update({_id: 2}, {$set: {"fruits.1": "Mango"}})
```

Add to array

```
db.student.update({_id: 1}, {$push: {subject: "History"}})
```

Remove from array

```
db.student.update({_id: 1}, {$pull: {subject: "Math"}})
```

◇ Embedded Documents

Insert document with nested fields

```
db.student.insert({
  rollno: 1,
  name: {fname: "Arya", lname: "Sharma"},
  marks: 30
})
```

Query nested field

```
db.student.find({"name.lname": "Sharma"})
```

◇ Date Queries

Insert date fields

```
db.student.insert({bdate: ISODate("2022-01-01"), name: "Arya"})
```

Find by date condition

```
db.student.find({bdate: {$lt: ISODate("2023-01-01")}})
```

◇ Field Modifiers

Increment a value

```
db.emp.update({empno: 4}, {$inc: {salary: 1000}})
```

Multiply a value

```
db.emp.update({empno: 4}, {$mul: {salary: 2}})
```

◇ Aggregation Framework

Group by and count

```
db.emp.aggregate([
  {$match: {salary: {$gt: 20000}}},
  {$group: {_id: "$dname", count: {$sum: 1}}}
])
```

Average quantity in category

```
db.product.aggregate([
  {$match: {category: "grocery"}},
  {$group: {_id: "$category", avgQty: {$avg: "$quantity"}}}
])
```

Total products below price

```
db.product.aggregate([
  {$match: {price: {$lt: 500}}},
  {$group: {_id: "$category", total: {$sum: 1}}}
])
```

◇ MapReduce (Advanced Use)

Map function

```
var map = function() {
  emit(this.dname, this.salary);
}
```

Reduce function

```
var reduce = function(key, values) {
  return Math.max.apply(Math, values);
}
```

Run MapReduce

```
db.emp.mapReduce(map, reduce, {
  out: "result",
  query: {city: "Ahmedabad"}
})
```

◇ Indexing

Create an index

```
db.student.createIndex({name: 1})
```

List indexes

```
db.student.getIndexes()
```

Use index hint

```
db.student.find({name: "Arya"}).hint({name: 1})
```

Explain query plan

```
db.student.find({name: "Arya"}).hint({name: 1}).explain()
```